

Learn ESP32 for Home Automation

Lesson 3: Serial Communication and Debugging

Serial Communication and Debugging – Study Notes

Key Concepts

1. **Serial communication basics** – how the Arduino (or other MCU) exchanges data with a computer via UART/USB.
2. **Using the Serial Monitor** – opening the monitor, setting the correct baud rate, and interpreting the output.
3. **Sending sensor data** – formatting, printing, and streaming real time measurements.
4. **Common debugging errors** – mismatched baud rates, missing `Serial.begin()`, buffer overflows, and wiring issues.
5. **Best practices for reliable debugging** – adding delimiters, using `Serial.flush()`, and keeping the loop non blocking.

Summary

Serial communication is the most straightforward way to interact with a microcontroller during development. By initializing the serial port with `Serial.begin(baudRate)`, the board can send text (or binary) data to the host PC over the USB cable. The Arduino IDE's Serial Monitor (or any terminal program) receives this stream, allowing you to view sensor readings, status messages, and error reports in real time.

When sending sensor data, it's important to format the output so it can be parsed later—using commas, tabs, or newline characters as delimiters. Debugging common pitfalls such as forgetting to start the serial interface, using a different baud rate on the monitor, or flooding the buffer with too much data helps keep sketches responsive. Applying simple techniques like printing only when values change, adding brief `delay()` or `yield()`, and checking wiring connections will make troubleshooting faster and more effective.

Important Points

- **Initialize once:** `Serial.begin(9600);` (or any supported baud rate) in `setup()`.
- **Match baud rates:** The number in the Serial Monitor must equal the rate set in `Serial.begin()`.
- **Use `Serial.println()` for readable lines** (adds `\r\n`). `Serial.print()` prints without a newline.
- **Delimiters:** CSV (`'value1,value2'`) or tab separated values are easy to import into spreadsheets.
- **Buffer size:** The Arduino's serial buffer is ~64 /bytes. Sending large strings without pauses can overflow it, causing lost characters.
 - **Common errors:**
 - No output !' missing `Serial.begin()` or wrong COM port.
 - Garbled characters !' mismatched baud rate.
 - Stopped output after a while !' buffer overflow or `while(!Serial);` blocking code.

- **Debugging tip:** Print short, descriptive messages (e.g., `Serial.println("Temp read OK");`) to pinpoint where the code stops.
- **Non blocking loops:** Avoid `delay()` in time critical code; use `millis()` to schedule sensor reads and serial prints.
- **`Serial.flush()`** (Arduino 1.0+): Waits for outgoing data to be transmitted; useful before resetting or entering low power mode.

Examples

1. Streaming Temperature Sensor Data (CSV)

```

`cpp
const int tempPin = A0;      // LM35 on analog pin A0
unsigned long lastPrint = 0;
const unsigned long interval = 500; // ms

void setup() {
  Serial.begin(9600);        // start serial at 9600 baud
  pinMode(tempPin, INPUT);
}

void loop() {
  if (millis() - lastPrint >= interval) {
    int raw = analogRead(tempPin);
    float voltage = raw * (5.0 / 1023.0);
    float tempC = voltage * 100.0; // LM35: 10 /mV/°C
    // CSV: timestamp,temperature
    Serial.print(millis());
    Serial.print(',');
    Serial.println(tempC, 2); // 2 decimal places
    lastPrint = millis();
  }
}
`

```

***Explanation:** The sketch reads an LM35 temperature sensor every 500 /ms and prints a line like ``12345,23.56``. The comma makes it easy to import into Excel or Python for plotting.

2. Debugging a Faulty Sensor Connection

```

`cpp
void setup() {
  Serial.begin(9600);
  Serial.println("=== Startup ===");
  pinMode(A1, INPUT);
}

void loop() {
  int val = analogRead(A1);

```

```

if (val == 0) {
  Serial.println("Warning: Sensor reads 0 – check wiring!");
} else {
  Serial.print("Sensor value: ");
  Serial.println(val);
}
delay(200); // simple pacing for demo
}
...

```

***Explanation:** The program prints a warning if the analog reading stays at 0, a common sign of a disconnected or broken sensor. By adding a clear message, you can quickly locate the problem without needing an oscilloscope.

Quick Review

1. ****What must you do in `setup()` before using `Serial.print()`?**
2. ****Why is it important to match the baud rate in the Serial Monitor with the one set in `Serial.begin()`?**
3. ****Name two symptoms of a serial buffer overflow and how to avoid it.**
4. ****How would you format multiple sensor values on a single line for easy CSV import?**
5. ****What debugging message would you add to verify that your sketch has reached a specific point in the code?**

Further Reading

- ****UART fundamentals**** – voltage levels, start/stop bits, and parity.
- ****Serial communication libraries**** – `SoftwareSerial`, `AltSoftSerial`, and `HardwareSerial` on various boards.
- ****Parsing serial data in Python**** – using `pySerial` and `pandas` for real time logging.
- ****Advanced debugging tools**** – logic analyzers, Serial Plotter, and ESP Home's OTA logging.
- ****Power efficient serial usage**** – sleeping the MCU while keeping the serial buffer intact.

